Assignment 1 - Git & GitHub Fundamentals

Session 1 - Group B | Individual Lab Report | October 13, 2026

Full Name: Mouaad Narjis
Student ID: 159916
Course Code & Section: CSC332402 - Software Engineering I
GitHub Username: Mn99971

Repository: https://github.com/Mn99971/git-session1-Mn99971

1. Repository Setup

I initialized the project with `git init -b main`. My .gitignore contains local.env, *.log, and !important.log. These rules ignore the local environment file and ordinary log files while keeping important.log available to Git. I retained sample.env as the starter example.

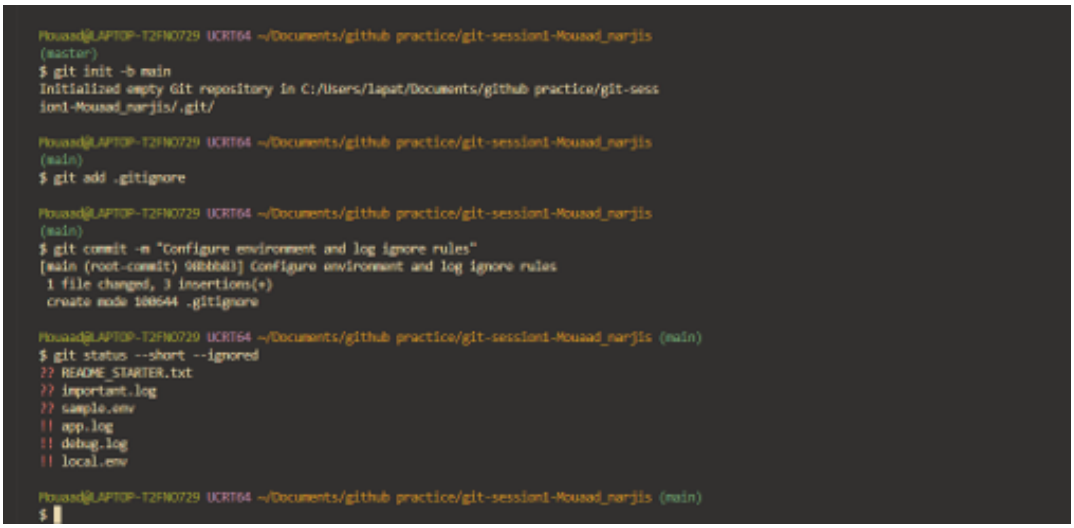


Figure 1. Repository initialization, first commit, and ignore state. The !! markers identify ignored files; important.log remains untracked and available to add.

2. README & Commit History

My README includes my name, GitHub username, project description, technologies, a code block, a link, and an image referenced as Report1.png. I committed both the README and image. I organized five local commits around ignore rules, starter documentation, student details, log verification, and the About feature, then added a separate remote README update.

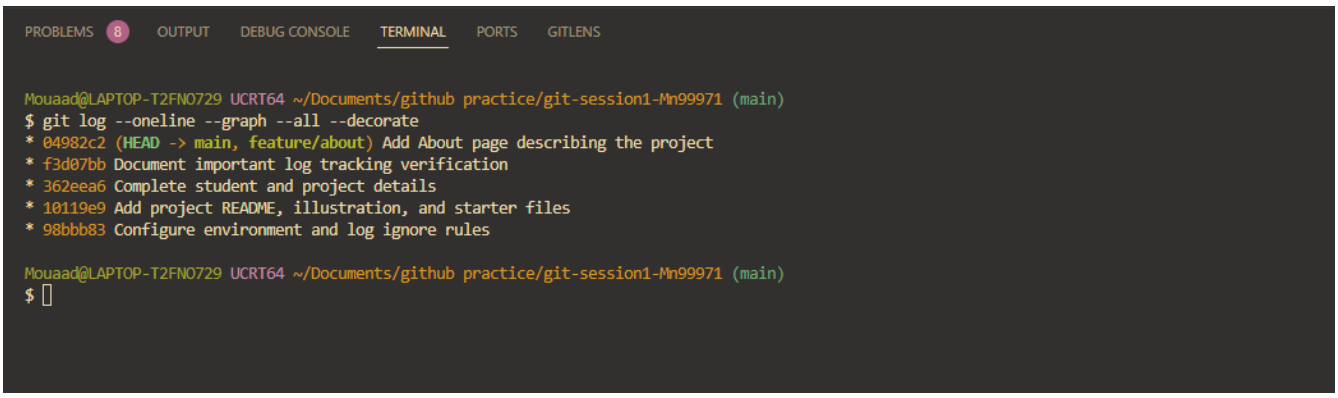


Figure 2. Five meaningful local commits before the GitHub README edit. main and feature/about point to the About commit after the merge.

3. Selective Staging

I modified README_STARTER.txt and important.log, then used git diff to inspect the unstaged changes. I staged only README_STARTER.txt using git add README_STARTER.txt and inspected the staged changes with git diff --staged. Git status distinguished the staged README starter from the unstaged log. I committed the student details first, leaving important.log modified, then committed the log separately.

```
Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$ git add README.md Report1.png README_STARTER.txt sample.env important.log

Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$ git commit -m "Add project README, illustration, and starter files"
[main 10119e9] Add project README, illustration, and starter files
 5 files changed, 39 insertions(+)
 create mode 100644 README.md
 create mode 100644 README_STARTER.txt
 create mode 100644 Report1.png
 create mode 100644 important.log
 create mode 100644 sample.env

Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$ git diff
diff --git a/important.log b/important.log
index 636e218..c685d17 100644
--- a/important.log
+++ b/important.log
@@ -1 +1,2 @@
-Important log entry.

Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (mai
n)
$ git add README_STARTER.txt
```

Figure 3. The unstaged diff for important.log after editing tracked files.

```
Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (mai
n)
$ git add README_STARTER.txt

Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
n)
$ git diff --staged
diff --git a/README_STARTER.txt b/README_STARTER.txt                              n)
index ea0037d..0be313e 100644
--- a/README_STARTER.txt
+++ b/README_STARTER.txt
@@ -1,5 +1,5 @@
 Git Lab Practice 1

-Student:
-Project:
-Technologies:
\ No newline at end of file
+Student: Mouaad Narjis
+Project:Stagera
+Technologies: Openxr, Unity, MetaSDK
\ No newline at end of file
```

Figure 4. The staged diff for README_STARTER.txt, shown using git diff --staged.

3. Selective Staging - Evidence Continued

```
Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$ t status
bash: t: command not found

Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README_STARTER.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   important.log

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Report2.png


Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$
```

Figure 5. Selective staging: README_STARTER.txt is staged, while important.log remains unstaged. A terminal typing mistake is also visible; it was not a repository or merge failure.

```
Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$ git commit -m "Complete student and project details"
[main 362eea6] Complete student and project details
 1 file changed, 3 insertions(+), 3 deletions(-)

Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   important.log

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Report2.png
        Report3.png
        Report4.png

no changes added to commit (use "git add" and/or "git commit -a")

Mouaad@LAPTOP-T2FNO729 UCRT64 ~/Documents/github practice/git-session1-Mn99971 (main)
$
```

Figure 6. After committing README_STARTER.txt, important.log remains modified and unstaged.

4. Branch & Merge

I created feature/about with git checkout -b feature/about, added ABOUT.md, and committed it on that branch. I returned to main using git checkout main and ran git merge feature/about. The history shows both branches pointing to the About commit, consistent with a fast-forward merge. I inspected the result with git log --oneline --graph --all --decorate (Figure 2).



Figure 7. About feature committed on feature/about, followed by checkout main and a successful fast-forward merge.

5. Remote Changes

I created an empty GitHub repository, connected it with git remote add origin, and published main using git push -u origin main. A commit records a staged snapshot in the local repository; push sends local commits to the remote repository.

I then edited README.md directly on GitHub. I ran git fetch origin first and used git status and git log --oneline main..origin/main to inspect the remote update. Fetch downloads remote objects and updates remote-tracking references without integrating the changes into my local files. The screenshot shows local main one commit behind origin/main. I then ran git pull to fetch and integrate the update into my local branch.

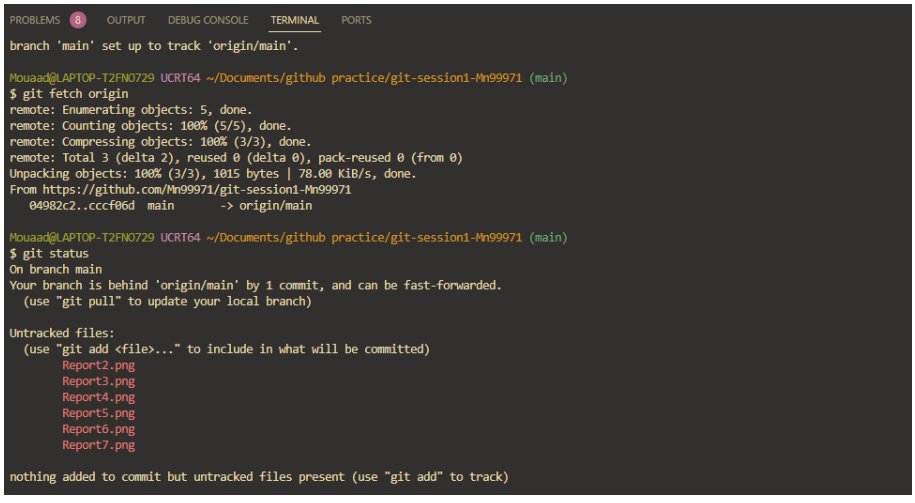


Figure 8. After fetch and before pull, local main is behind origin/main by one commit and can be fast-forwarded. Additional screenshot files are untracked at this point.

Troubleshooting: I encountered no Git workflow errors. A typing mistake in a terminal command produced a command-not-found message (Figure 5). I corrected the command and continued; no repository recovery was necessary.

6. Fair Use of AI

I used ChatGPT to break down the assignment, suggest a sequence of Git commands, explain image references in Markdown, and help assemble this report. My questions included whether the branch should be named main, whether the environment file should be local.env, how to add an image, how to switch branches, and how to merge using the commands covered in class. I accepted the basic workflow and README example, but chose to create and edit files directly in VS Code instead of using shell commands. I also used git checkout and the basic git merge command rather than the suggested git switch and --no-ff option. I checked the instructions against the assignment requirements and verified the results through terminal output, status, diffs, and commit history. AI organized the report; the reflection below is my own paragraph, with minor wording and spelling edits.

7. Sources & Reflection

Sources used:
Assignment 1 and Report Template - group B.pdf and the supplied starter files.
ChatGPT guidance during this lab, disclosed above.
My terminal output and screenshots as evidence of the work.

Student reflection: I was already familiar with GitHub before this assignment, but using Git itself was a fun experience. There was not much new to me about what a commit or branch is, or what gitignore and .env files are for. What I did learn was the difference between fetching and pulling, and how we can use fetch to preview remote changes before integrating them when someone else is working on the same part of a project. Fetch does not integrate changes itself; pulling may require resolving conflicts. I mostly used GitHub/Git for managing my own projects and version control, so that ambiguity did not really come up because I worked alone.

First diagnostic check: If Git behaves unexpectedly, I would first run git status to check the current branch and whether files are staged, unstaged, or untracked. I would then inspect the relevant diff and commit history before changing anything.